

Lo primero fue preparar la base de datos. Hice un archivo db.json y le añadí un servidor de prueba. Después, arranqué el servidor desde la terminal con el comando json-server para que todo funcionara y se conectara por el puerto 3000.

Visualmente, usé Bootstrap para que la página se viera limpia y moderna. La dividí en dos columnas, para que la visualización de los servidores registrados fuera más cómoda. En la columna de la izquierda, coloqué un formulario donde se puede introducir el nombre del servidor, cuantos núcleos serían necesarios, la RAM, el almacenamiento y el presupuesto que maneja. La columna de la derecha la dejé para que ahí aparecieran las tarjetas de los servidores que se fueran registrando.

Todo el funcionamiento lo programé en el archivo app.js. Lo hice así:

Para mostrar los datos (**GET**): Cuando la página se carga, el código va al servidor, recoge los servidores y los dibuja en la pantalla. Añadí un detalle visual: si el servidor tiene el almacenamiento HDD, el borde de la tarjeta sale con un tono azul, y, por el contrario, si el almacenamiento es SSD, el borde será verde.

Para guardar visitantes nuevos (**POST**): Al rellenar el formulario y pulsar el botón de registrar, JavaScript toma esos datos y los envía al archivo db.json para que queden guardados. Además, el formulario se vacía y la nueva tarjeta aparece en la derecha enseguida, sin que la página tenga que recargarse.

Para borrar visitantes (**DELETE**): Cada tarjeta lleva un botón de "Eliminar". Al hacer clic en él, el servidor en concreto se borra por completo del fichero y de la pantalla en ese mismo instante.

Para el despliegue (Docker y Render): Finalmente, creé un archivo Dockerfile en la raíz para empaquetar toda la aplicación usando Node.js e instalando json-server de forma global. Subí el proyecto a GitHub y lo conecté con la plataforma Render, que detectó el Dockerfile automáticamente y subió la base de datos a internet. Para acabar, cambié la dirección local de la API en app.js por la nueva URL pública de Render para que todo funcione conectado en la nube.